МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра ИИТ

Отчет по лабораторной работе №7

Специальность ПО-12

Выполнил:
Студент 3 курса
Группы ПО-12
Мельник Р .С.
Проверила:
Кулик А. Д..

Брест 2026

Цель работы: освоить возможности языка программирования Python в

разработке оконных приложений

Задание 1: Построение графических примитивов и надписей
Вариант 9: Создать классы Point и Line. Объявить список из n объектов класса Point. Для объекта класса Line определить, какие из объектов Point лежат на одной стороне от прямой линии и какие на другой. Реализовать ввод данных для объекта Line и случайное задание данных для объекта Point.

Код программы:

```
import tkinter as tk
from tkinter import messagebox
import random
import math
from PIL import ImageGrab
import os
import datetime

class Point:
    def __init__(self, x, y, dx, dy):
        self.x = x
        self.y = y
        self.dx = dx
        self.dy = dy

class Line:
    def __init__(self, x1, y1, x2, y2):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2

    def get_side(self, p: Point):
        return (p.x - self.x1) * (self.y2 - self.y1) - (p.y - self.y1) * (self.x2 - self.x1)

class Task1App:
```

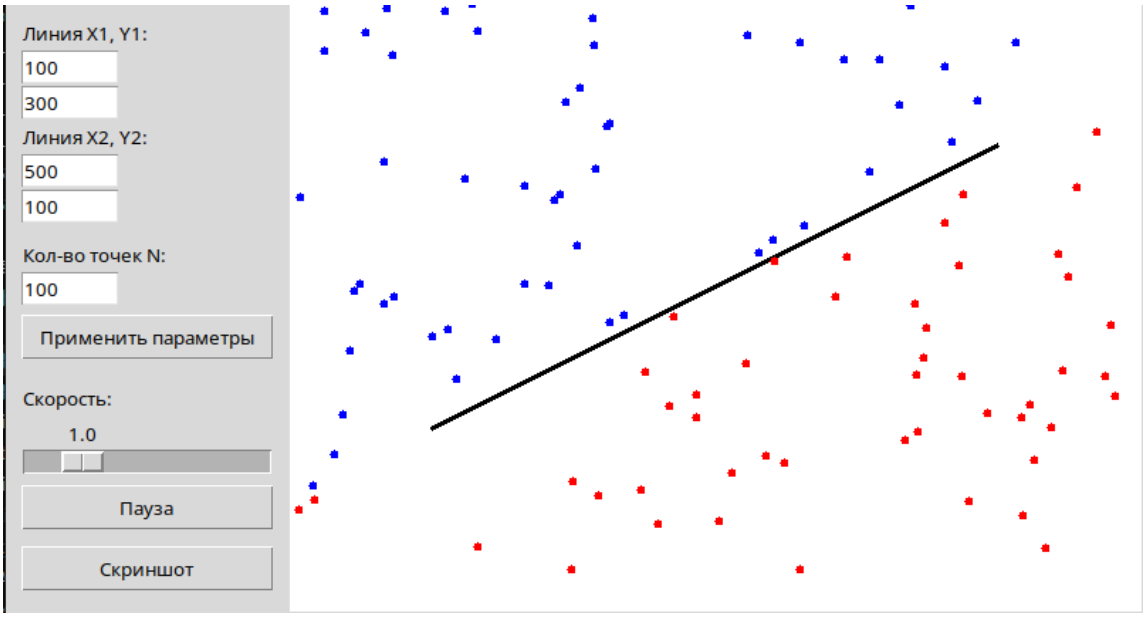
```
    def __init__(self, root):
        self.root = root
        self.root.title("ЛР 7 - Задание 1: Точки и Линия")

        self.width = 600
        self.height = 400
        self.points =[]
        self.is_paused = False

        control_frame = tk.Frame(root, padx=10, pady=10)
        control_frame.pack(side=tk.LEFT, fill=tk.Y)

        self.canvas = tk.Canvas(root, width=self.width, height=self.height, bg="white")
        self.canvas.pack(side=tk.RIGHT, expand=True, fill=tk.BOTH)

        tk.Label(control_frame, text="Линия X1, Y1:").pack(anchor=tk.W)
        self.ent_x1 = tk.Entry(control_frame, width=8); self.ent_x1.pack(anchor=tk.W);
self.ent_x1.insert(0, "100")
        self.ent_y1 = tk.Entry(control_frame, width=8); self.ent_y1.pack(anchor=tk.W);
self.ent_y1.insert(0, "300")

        tk.Label(control_frame, text="Линия X2, Y2:").pack(anchor=tk.W)
        self.ent_x2 = tk.Entry(control_frame, width=8); self.ent_x2.pack(anchor=tk.W);
self.ent_x2.insert(0, "500")
        self.ent_y2 = tk.Entry(control_frame, width=8); self.ent_y2.pack(anchor=tk.W);
self.ent_y2.insert(0, "100")

        tk.Label(control_frame, text="Кол-во точек N:").pack(anchor=tk.W, pady=(10,0))
        self.ent_n = tk.Entry(control_frame, width=8); self.ent_n.pack(anchor=tk.W);
self.ent_n.insert(0, "100")

        tk.Button(control_frame, text="Применить параметры",
command=self.apply_params).pack(fill=tk.X, pady=5)

        tk.Label(control_frame, text="Скорость:").pack(anchor=tk.W, pady=(10,0))
        self.speed_scale = tk.Scale(control_frame, from_=0.1, to=5.0, resolution=0.1,
orient=tk.HORIZONTAL)
```

```
        self.speed_scale.set(1.0)
        self.speed_scale.pack(fill=tk.X)

        self.btn_pause = tk.Button(control_frame, text="Пауза",
command=self.toggle_pause)
        self.btn_pause.pack(fill=tk.X, pady=5)

        tk.Button(control_frame, text="Скриншот",
command=self.take_screenshot).pack(fill=tk.X, pady=5)

        self.line = Line(100, 300, 500, 100)
        self.apply_params()
        self.update_animation()

    def apply_params(self):
        try:
            x1, y1 = float(self.ent_x1.get()), float(self.ent_y1.get())
            x2, y2 = float(self.ent_x2.get()), float(self.ent_y2.get())
            self.line = Line(x1, y1, x2, y2)

            n = int(self.ent_n.get())
            if len(self.points) != n:
                self.points =[]
                for _ in range(n):
                    x = random.randint(10, self.width - 10)
                    y = random.randint(10, self.height - 10)
                    dx = random.choice([-1, 1]) * random.random() * 2
                    dy = random.choice([-1, 1]) * random.random() * 2
                    self.points.append(Point(x, y, dx, dy))
        except ValueError:
            messagebox.showerror("Ошибка", "Введите корректные числа")

    def toggle_pause(self):
        self.is_paused = not self.is_paused
        self.btn_pause.config(text="Возобновить" if self.is_paused else "Пауза")

    def take_screenshot(self):
```

```
        x = self.root.winfo_rootx() + self.canvas.winfo_x()
        y = self.root.winfo_rooty() + self.canvas.winfo_y()
        w = self.canvas.winfo_width()
        h = self.canvas.winfo_height()

        try:
            img = ImageGrab.grab(bbox=(x, y, x+w, y+h))
            filename =
f"screenshot_task1_{datetime.datetime.now().strftime('%H%M%S')}.png"
            img.save(filename)
            messagebox.showinfo("Скриншот", f"Сохранено в
директорию:\n{os.path.abspath(filename)}")
        except Exception as e:
            messagebox.showerror("Ошибка скриншота", str(e))

    def update_animation(self):
        if not self.is_paused:
            speed = self.speed_scale.get()
            self.canvas.delete("all")

            self.canvas.create_line(self.line.x1, self.line.y1, self.line.x2, self.line.y2,
fill="black", width=3)

            for p in self.points:
                p.x += p.dx * speed
                p.y += p.dy * speed

                if p.x < 0 or p.x > self.width: p.dx *= -1
                if p.y < 0 or p.y > self.height: p.dy *= -1

                side = self.line.get_side(p)
                color = "blue" if side > 0 else ("red" if side < 0 else "green")

                self.canvas.create_oval(p.x-3, p.y-3, p.x+3, p.y+3, fill=color, outline="")

        self.root.after(20, self.update_animation)
```

```
if __name__ == "__main__":
    root = tk.Tk()
    app = Task1App(root)
    root.mainloop()
```

Результат:



Задание 2: Реализовать построение заданного типа фрактала по варианту
Вариант 9: Остров Минковского

Код программы:

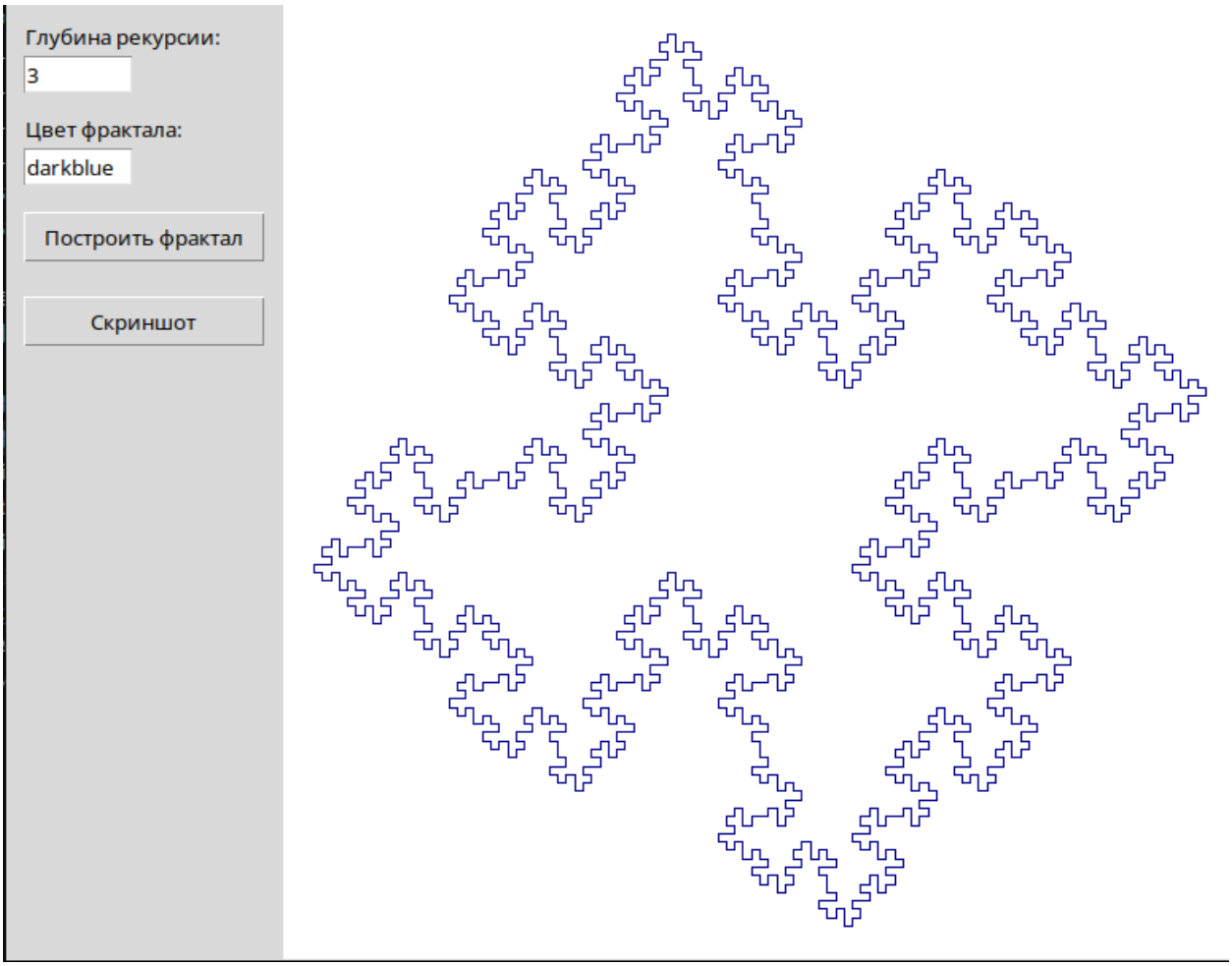
```
import tkinter as tk
from tkinter import messagebox
import math
from PIL import ImageGrab
import os
import datetime

class Task2App:
    def __init__(self, root):
        self.root = root
        self.root.title("ЛР 7 - Задание 2: Фрактал Остров Минковского")
```

```
        self.width = 600
        self.height = 600

        control_frame = tk.Frame(root, padx=10, pady=10)
        control_frame.pack(side=tk.LEFT, fill=tk.Y)

        self.canvas = tk.Canvas(root, width=self.width, height=self.height, bg="white")
        self.canvas.pack(side=tk.RIGHT, expand=True, fill=tk.BOTH)

        tk.Label(control_frame, text="Глубина рекурсии:").pack(anchor=tk.W)
        self.ent_depth = tk.Entry(control_frame, width=8)
        self.ent_depth.pack(anchor=tk.W)
        self.ent_depth.insert(0, "3")

        tk.Label(control_frame, text="Цвет фрактала:").pack(anchor=tk.W,
pady=(10,0))
        self.ent_color = tk.Entry(control_frame, width=8)
        self.ent_color.pack(anchor=tk.W)
        self.ent_color.insert(0, "darkblue")

        tk.Button(control_frame, text="Построить фрактал",
command=self.draw_fractal).pack(fill=tk.X, pady=15)
        tk.Button(control_frame, text="Скриншот",
command=self.take_screenshot).pack(fill=tk.X, pady=5)

        self.draw_fractal()

    def generate_l_system(self, depth):
        path = "F+F+F+F"
        rule = "F-F+F+FF-F-F+F"

        for _ in range(depth):
            path = path.replace("F", rule)
        return path

    def draw_fractal(self):
        try:
```

```
        depth = int(self.ent_depth.get())
        if depth < 0 or depth > 5:
            messagebox.showwarning("Внимание", "Глубина от 0 до 5, иначе будет
зависание!")
            return

        color = self.ent_color.get()
        self.canvas.delete("all")

        path = self.generate_l_system(depth)

        coords = [(0, 0)]
        x, y = 0, 0
        angle = 0.0

        for cmd in path:
            if cmd == 'F':
                x += math.cos(angle)
                y += math.sin(angle)
                coords.append((x, y))
            elif cmd == '+':
                angle += math.pi / 2
            elif cmd == '-':
                angle -= math.pi / 2

        min_x = min(c[0] for c in coords)
        max_x = max(c[0] for c in coords)
        min_y = min(c[1] for c in coords)
        max_y = max(c[1] for c in coords)

        fractal_width = max_x - min_x
        fractal_height = max_y - min_y

        padding = 40
        scale = min((self.width - padding) / fractal_width,
                    (self.height - padding) / fractal_height)
```

```
            offset_x = (self.width - fractal_width * scale) / 2 - min_x * scale
            offset_y = (self.height - fractal_height * scale) / 2 - min_y * scale

            screen_coords =[]
            for cx, cy in coords:
                screen_coords.extend([cx * scale + offset_x, cy * scale + offset_y])

            self.canvas.create_line(*screen_coords, fill=color, width=1)

        except ValueError:
            messagebox.showerror("Ошибка", "Неверно заданы параметры")

    def take_screenshot(self):
        x = self.root.winfo_rootx() + self.canvas.winfo_x()
        y = self.root.winfo_rooty() + self.canvas.winfo_y()
        w = self.canvas.winfo_width()
        h = self.canvas.winfo_height()

        try:
            img = ImageGrab.grab(bbox=(x, y, x+w, y+h))
            filename =
f"screenshot_task2_{datetime.datetime.now().strftime('%H%M%S')}.png"
            img.save(filename)
            messagebox.showinfo("Скриншот", f"Сохранено в
директорию:\n{os.path.abspath(filename)}")
        except Exception as e:
            messagebox.showerror("Ошибка скриншота", str(e))

if __name__ == "__main__":
    root = tk.Tk()
    app = Task2App(root)
    root.mainloop()
```

Результат:



Вывод: освоили возможности языка программирования Python в разработке оконных приложений